

WISER-Nestlé DOM optimizer: business and technical summary

Decision problem

Distributed Order Management (DOM) decides which distribution center (DC) should serve an order and when it should ship. The default DC may lack inventory or operational headroom, so a planner can divert the order to another eligible DC. A diversion may improve customer service and avoid shortage penalties, but it can also increase shipping cost, use scarce inventory needed by another order, or consume an alternate dock slot. The useful decision is therefore not “which DC looks best for this order?” It is “which compatible set of assignments produces the best validated network outcome?”

DOM is combinatorial. If each of O decision units has K possible outcomes, the assignment space grows approximately as K^O . Each assignment also determines integer SKU fulfillment quantities. Orders interact through DC-SKU inventory and DC-date resources, while orders on the same source load must move together. A locally attractive choice can prevent a more valuable combination later.

Supplied data and business rules

All ten challenge files are readable and pass a format-specific fail-fast audit. The input includes 25,193 order-SKU rows, 377,504 inventory/forecast rows, 12,922 shipping lanes, 480 dock rows, 530 throughput-utilization rows, a worked workbook, and the source equations document. The two supplied recommendation outputs contain 1,109 orders and 25,193 SKU rows. They reconcile to 2,554,440 requested and 2,413,937 selected fulfilled cases, or 94.4997% case fill. Only privacy-safe aggregates are reported.

The adapter derives cases from the source planning unit. It uses division by planning-units-per-case when that value is positive and converts pallet planning units using cases-per-pallet otherwise. This exactly reconciles source demand with the supplied output.

Inventory is protected available-to-promise (ATP): for a candidate ship date, usable inventory is the minimum nonnegative availability through the following five days. This protects future committed demand. Alternatives require every load SKU to be present at the candidate DC. Shipping lead is the ceiling of distance divided by 500 miles per day; alternative PGI is rolled backward over weekends and configured holidays. `Dock_Remaining` is a hard incremental alternate-load limit. The supplied throughput table contains observed utilization, not a documented maximum, so it is used only in explicitly labeled headroom scenarios.

Orders sharing a load select one common DC/date. Shipping and alternate dock use are charged once per load through a deterministic leader. A diversion must improve the default protected-ATP fill by at least both five percentage points of total demand and 100 cases, capped at demand.

The order penalty is not a simple linear shortage cost. It activates only when fill falls below the order’s threshold. When active, it includes variable unmet value, fixed cost, and cost per cut SKU, followed by optional minimum and maximum amounts. The implemented equation reproduces the supplied default penalty to floating-point tolerance.

Common optimization model

Every method maximizes the same independently recomputed objective:

$$\text{fulfilled value} - \text{thresholded penalty} - \text{shipping cost}.$$

The deterministic mixed-integer linear program (MILP) selects one eligible DC/PGI candidate or an unassigned outcome for each order. Partial fulfillment is allowed at the selected DC, but SKU lines cannot split across DCs. Constraints enforce demand balance, load cohesion, projected ATP at every checkpoint, dock and enabled scenario resources, exact full-pallet/loose-case accounting, and the diversion-uplift rule. SciPy submits the model to the HiGHS solver, which returns a feasible incumbent, mathematical bound, gap, model size, and runtime.

Two transparent baselines use the same rules. The **default baseline** keeps each load at its eligible default option and allocates shared resources deterministically. The **greedy baseline** considers alternatives sequentially, commits the best current incremental objective, and immediately updates residual inventory and capacity. It is fast and explainable but cannot undo an early myopic decision.

Why the hybrid split is appropriate

A monolithic QUBO containing every assignment, quantity, inventory slack, capacity slack, and thresholded-penalty auxiliary would be too wide and penalty-sensitive for current quantum hardware. This solver instead uses quantum or quantum-inspired search only for a bounded assignment neighborhood.

The hybrid starts from a feasible default or greedy incumbent. It selects complete load groups that contend for shared resources, creates a small one-hot QUBO over candidate plans, and samples combinations using exact enumeration, a random control, simulated annealing, or an optional approved D-Wave backend. Sample repair restores one common option per load. Exact local MILP recourse then chooses SKU quantities using resources left after frozen orders. The complete solution is independently validated and accepted only when it is a strict objective improvement.

This gives the invariant

$$J(S^{k+1}) \geq J(S^k).$$

Sampling quality affects search efficiency, not the correctness or minimum quality of the returned recommendation. The QUBO width is capped, so global order count can grow without requiring a proportionally larger quantum subproblem.

Evaluation and evidence

The runnable notebook implements the challenge comparison plus size scaling, penalty-weight sensitivity, candidate-count sensitivity, inventory shocks, four-seed coefficient-noise tests, Pareto-pruning ablation, random-versus-conflict batching, and random-versus-annealing sampling. An independent synthetic coordination control tests whether the hybrid can revisit a greedy trap.

The complete smoke profile produced 37 aggregate runs; every row was feasible and no hybrid result regressed its own default incumbent. On the real four-decision-unit comparison, the exact MILP proved a zero gap. Setting the default objective to index 100, greedy, exact MILP, and hybrid each reached 136.5; hybrid used a nine-variable local QUBO and one accepted move. This does not show a quantum advantage: greedy also reached the exact solution and the sampler was classical.

On the nine-order scaling subset created by whole-load expansion, greedy and exact MILP agreed while the one-iteration smoke hybrid improved its default start but did not catch them. This is useful negative evidence: the acceptance design remained safe while the search budget was insufficient. In the Pareto ablation, candidate rows fell from 12 to 6 with unchanged validated objective. The synthetic control improved greedy by 23 synthetic units but remained 249.4 units below the exact optimum, again showing limited search value rather than dominance.

Trade-offs and recommendation

The exact MILP is the quality reference on tractable subsets because a zero gap is a proof. Greedy is the operational low-latency fallback. The hybrid is appropriate when the global MILP is time-limited and planners want repeated, coordinated searches of the most resource-coupled neighborhoods with a safe incumbent.

Current noisy quantum hardware adds embedding, coefficient-range, queue, chain, and latency costs. Physical qubits are not equivalent to dense logical variables. A fair hardware study must keep the QUBO, warm start, repair, recourse, validator, seeds, and wall-clock boundary identical across samplers, and must report embedding and uncertainty. No QPU run or quantum advantage is claimed here.

The recommended deployment sequence is: retain the exact and greedy classical controls; run the full aggregate experiment profile inside the approved environment; review planner explanations for every recommended diversion; confirm the enterprise holiday calendar and throughput-limit semantics; and only then consider an approved QPU comparison. The aggregate-only copilot is useful for exploring evidence, but it is decision support—not an autonomous routing agent.